CEIMM

CENTRO DE ELECTRÓNICA INDUSTRIAL Y SISTEMAS MULTIMODALES

Wireless Sensor Networks Lab: Contiki-NG and Cookie platform

Gabriel Mujica
Jorge Portilla

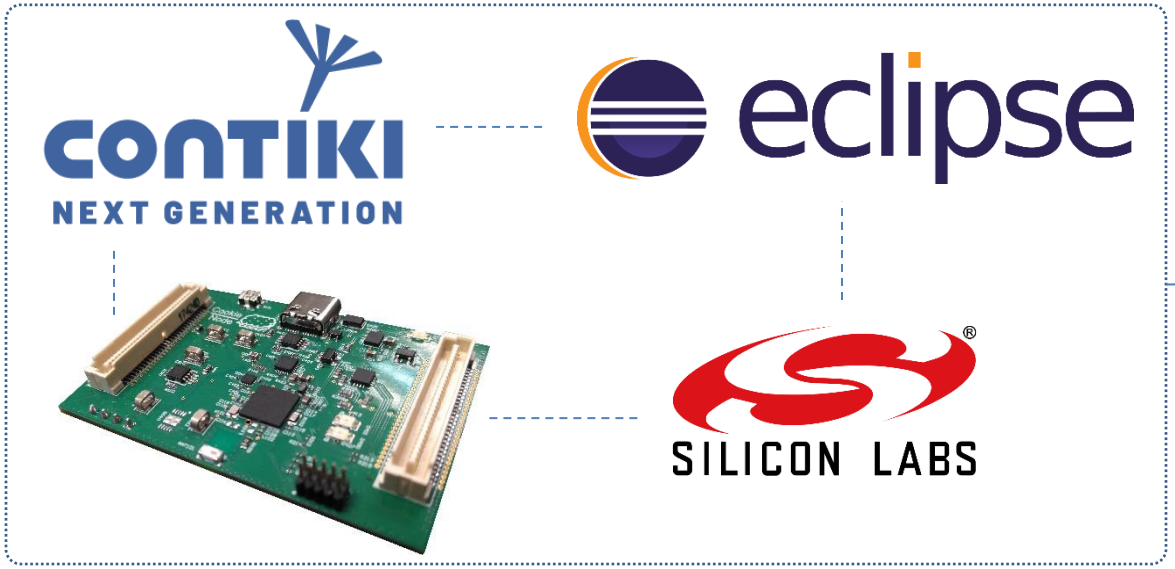
cei@upm.es

UNIVERSIDAD POLITÉCNICA DE MADRID

POLITÉCNICA

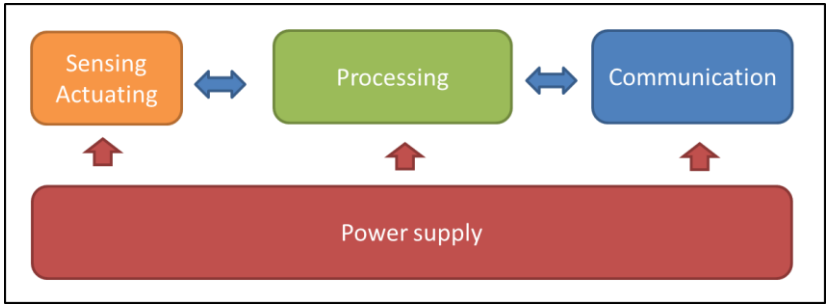
Main goals of the lab

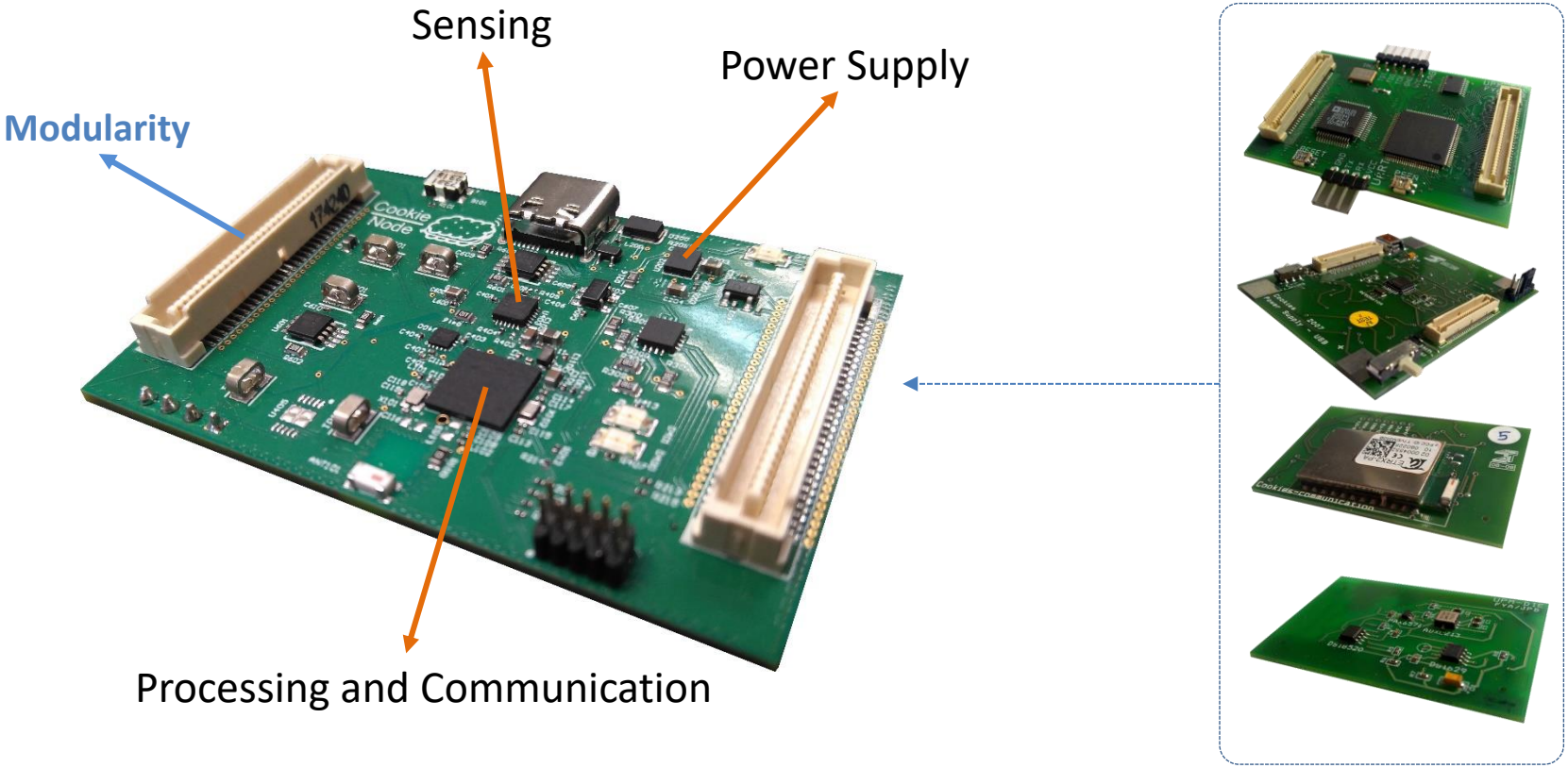
- Make use of the basic knowledge gained in the first lab, with a real WSN hardware platform.
- Start learning the basics of the Contiki-NG Operating System.
- Integration of hardware and software tools to provide a development ecosystem for prototyping.
- Combination of Contiki-NG with the Cookie nodes to create basic WSN-IoT sensing applications, as the baseline for the practical work of the subject.



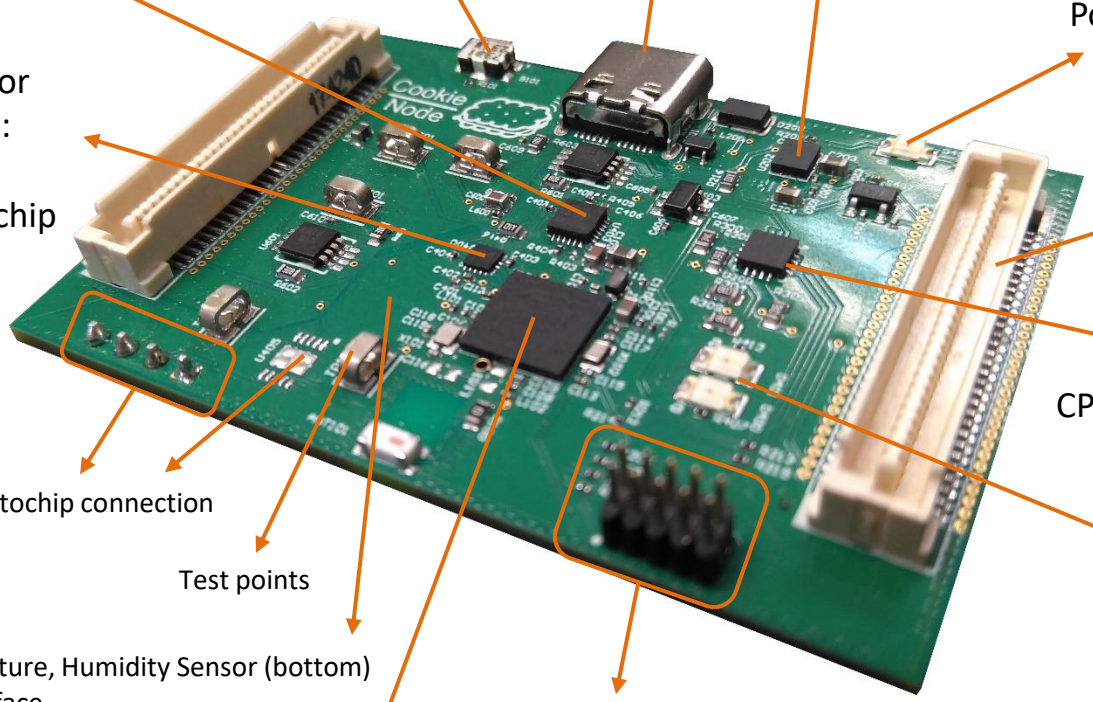
Hardware elements

- Cookie Platform:
  - Self-contained hardware node with all the elements that compose a WSN node





Hardware elements



Inertial Sensor
(3-axis accelerometer,
3-axis gyroscope)
I2C/SPI Interface

Analog Switches for
circuit activation:
Sensors, Power
monitoring, Cryptochip

USB-C (Power Supply and User Debugging/Interface)

Reset Button

Power conversion

Power LED

Vertical Connectors

CP2102 (USB to UART Bridge)

User LEDs

Cryptochip connection

Test points

Temperature, Humidity Sensor (bottom)
I2C Interface

Debugger/Bootloader Connector

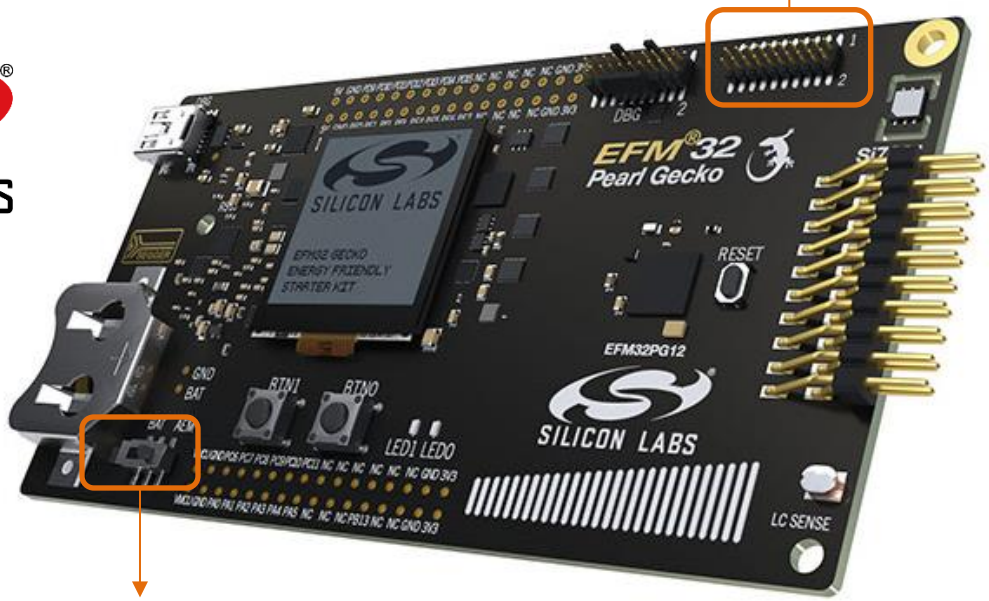
Silicon Labs **EFR32MG12** Low Power Wireless System-on-Chip
ARM Cortex-M4 MCU

Hardware elements

EFM32™ Pearl Gecko PG12 Starter Kit

Integrated Segger J-Link USB debugger
Advanced Energy Monitoring system
Built-in USB serial port

Connection to an external device

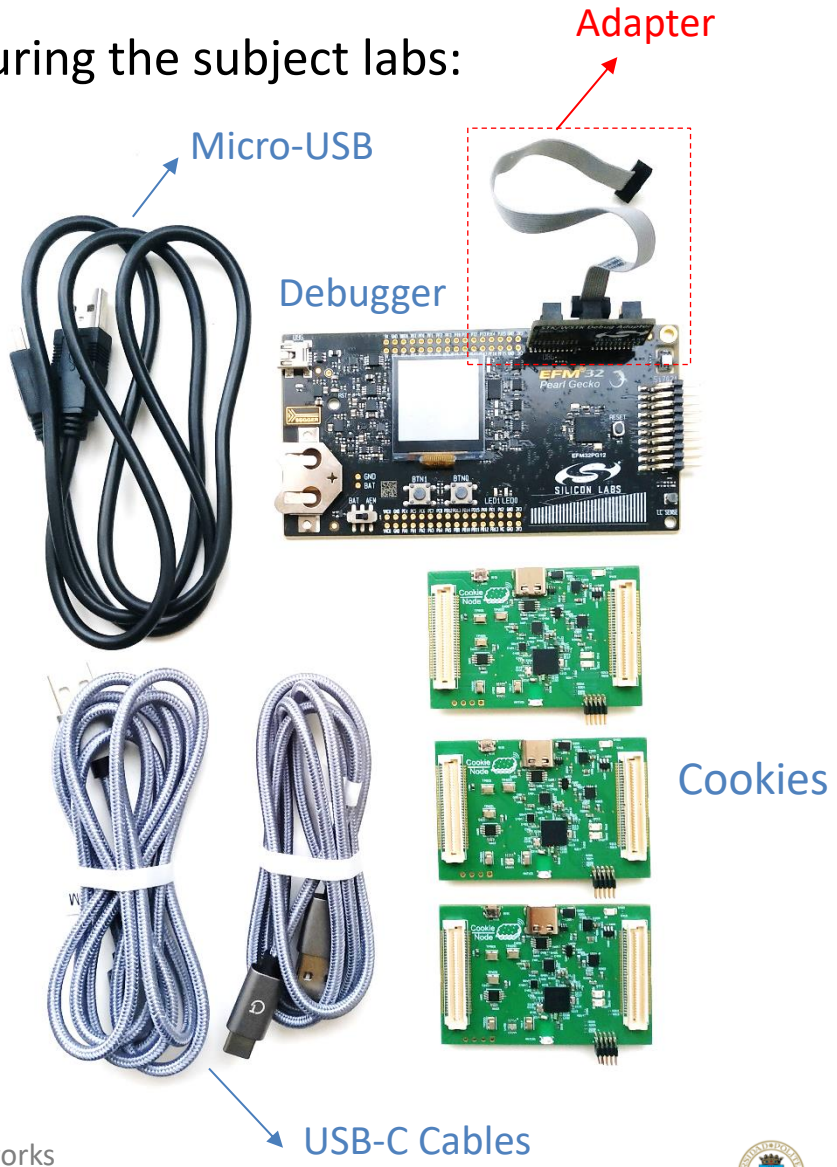


Make sure the switch is in **AEM** position

Hardware elements

WSN **Hardware Development Kit** to be used during the subject labs:

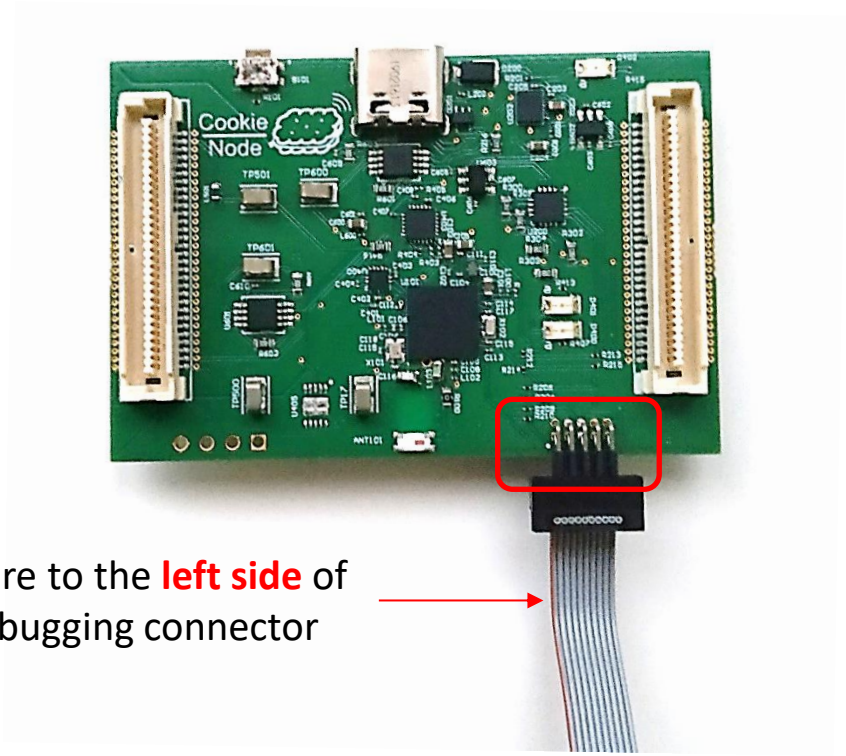
- 3 Cookie nodes
- 1 USB debugger + micro-USB cable
- 1 adapter + cable to connect the debugger with the Cookie node
- 2 USB-C cables for power + data



IMPORTANT: The Cookie nodes **can only be programmed with the debugger board**. Once the node is programmed, the debugger can be used to power supply the node and also as serial interface bridge (i.e. power + data).
The other 2 Cookie nodes can be power-supplied through the USB cable, but it is not possible to program them with it. **They have to be previously programmed** with the debugger board.
Do not connect the **debugger and the USB-C** at the same time!

Hardware elements

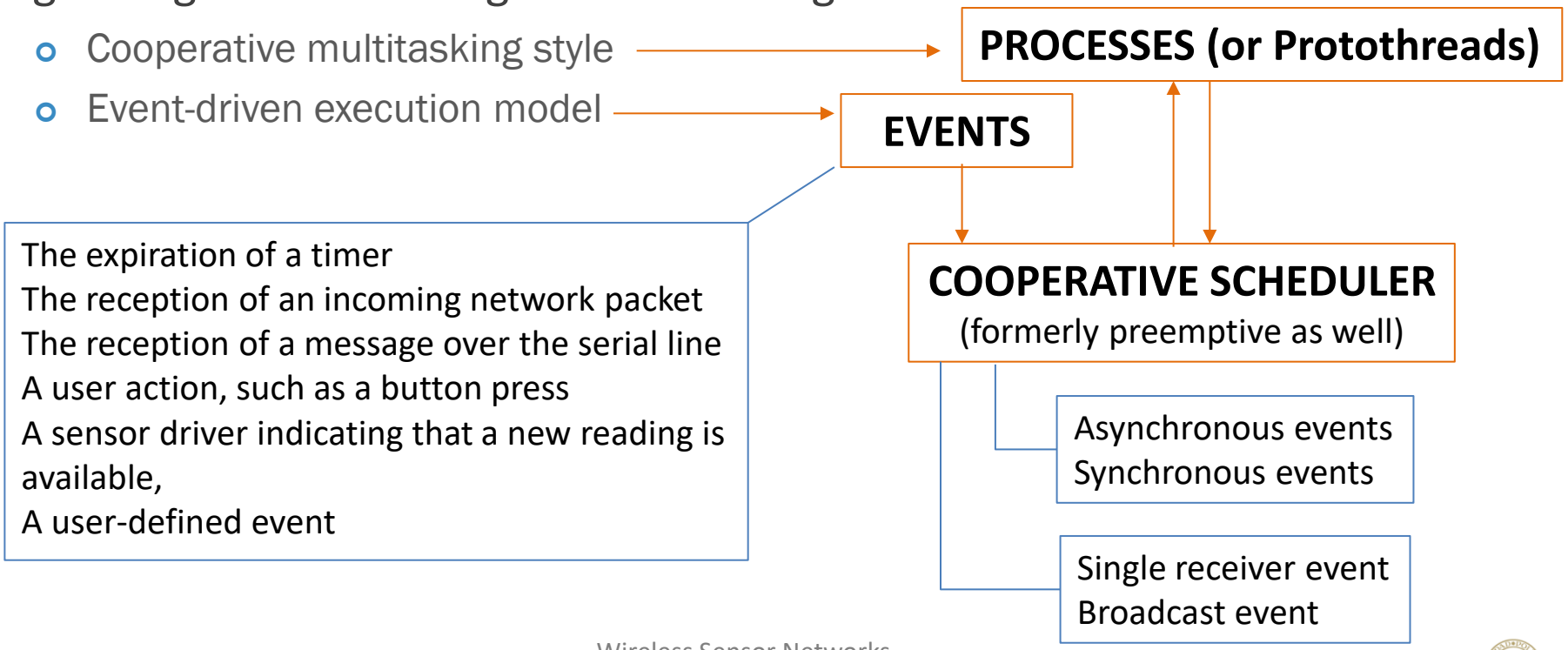
Connection of the USB Debugger with the Cookie node:



Red wire to the **left side** of the debugging connector

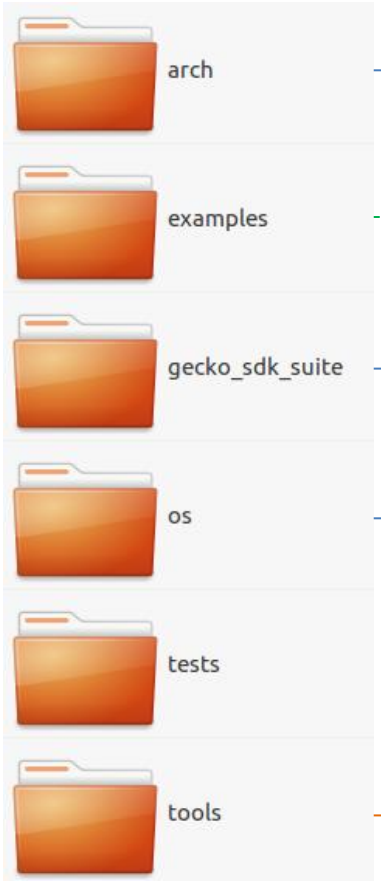
Contiki-NG

- Operating system for resource-constrained devices for IoT
- RFC-compliant, low-power IPv6 communication stack
- Energy-efficient architectures such as the ARM Cortex-M3/M4 (and the traditional Texas Instruments MSP430)
- Lightweight Multitasking and Scheduling
  - Cooperative multitasking style
  - Event-driven execution model



Contiki-NG

- Repository structure of Contiki-NG



Hardware-dependent Code and HAL. Specific support for different platforms and architectures (including the Cookie node)



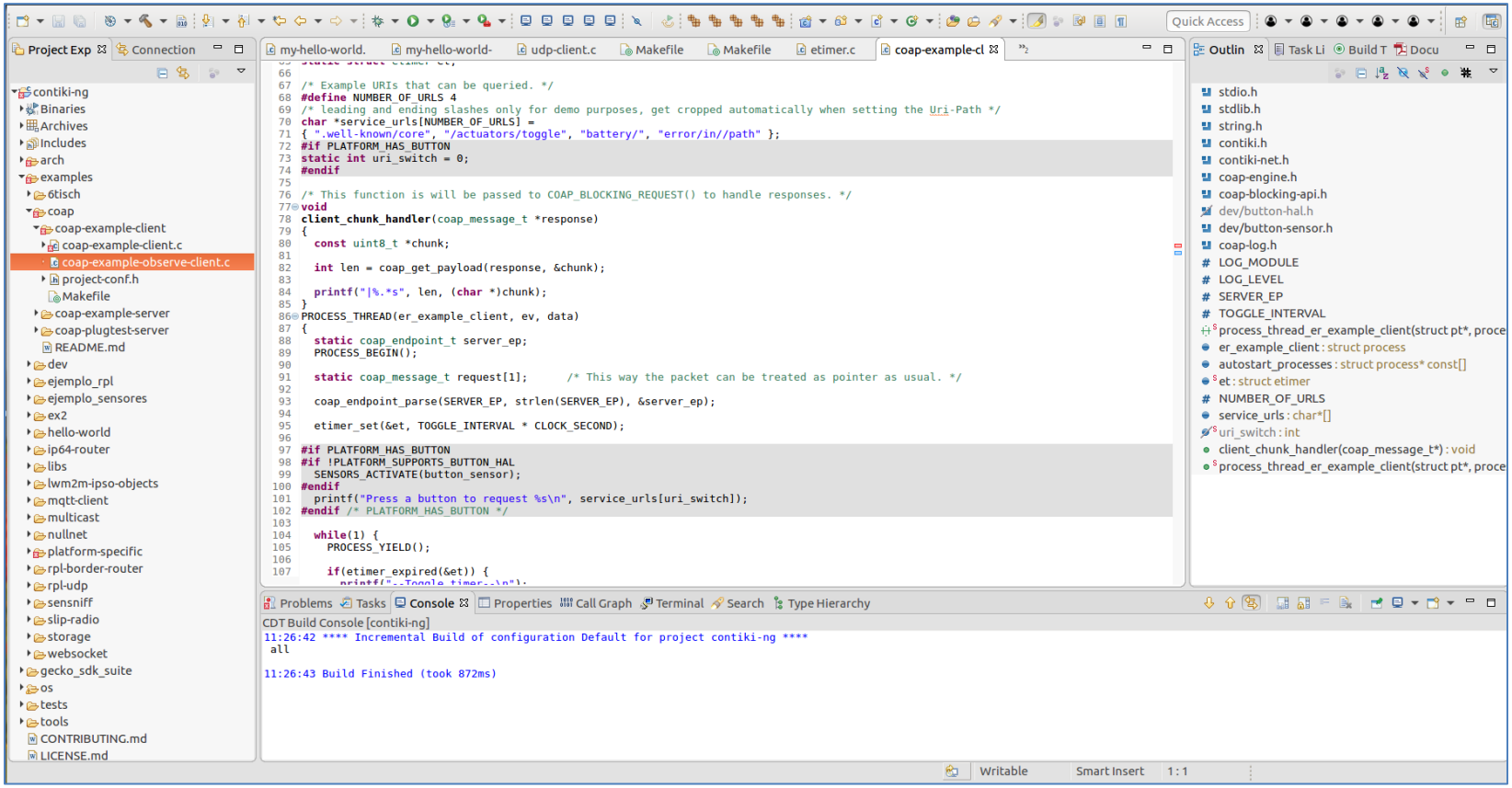
Contiki-NG code. Definition of primitives, processes, timers, networking stack, etc.

User/Application space

Supported tools outside the embedded code (such as Cooja)

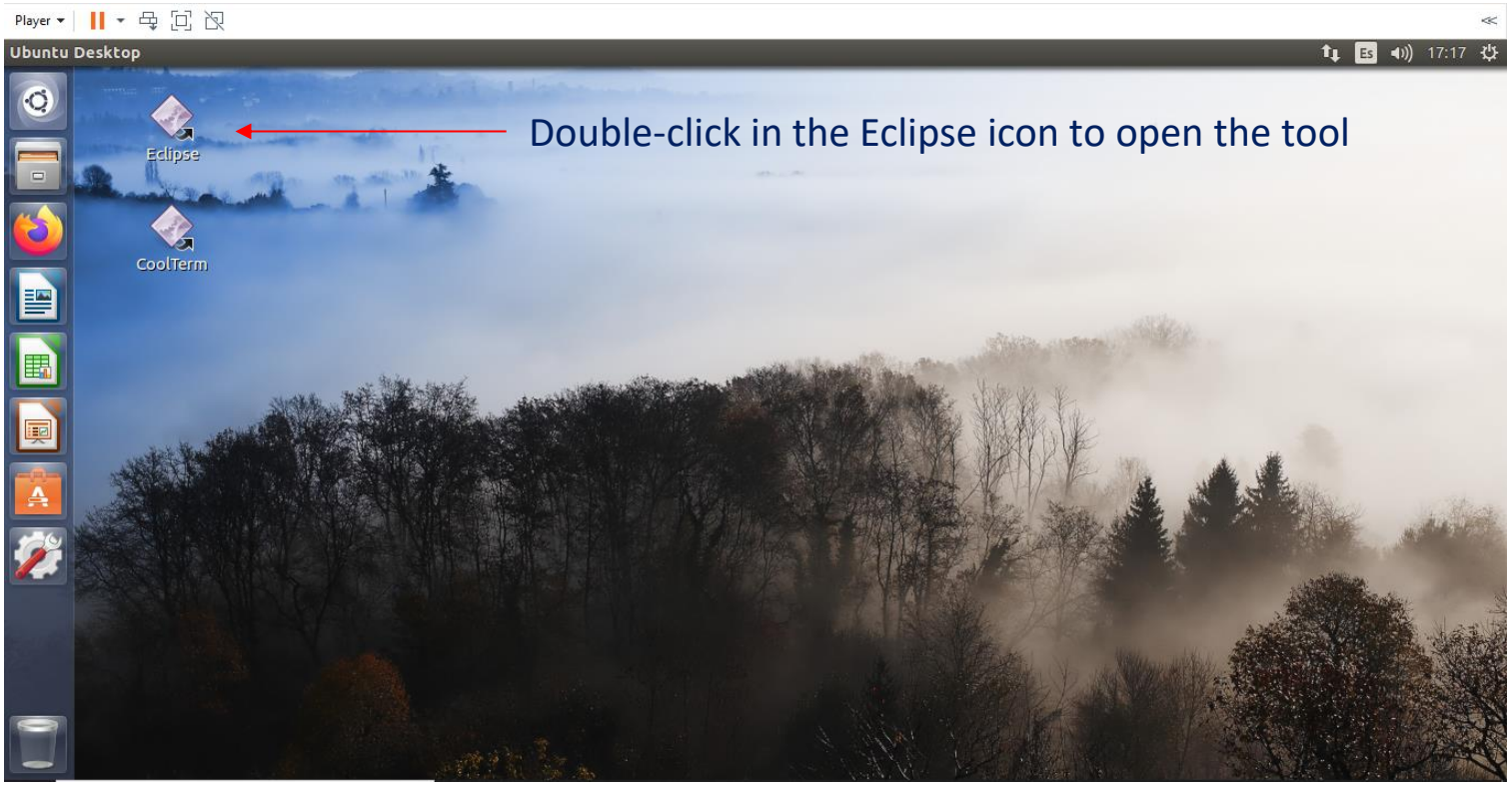
Eclipse Integration

- Eclipse Open-source Development Environment
  - Integration of Contiki-NG into the tool to speed up the code and application development



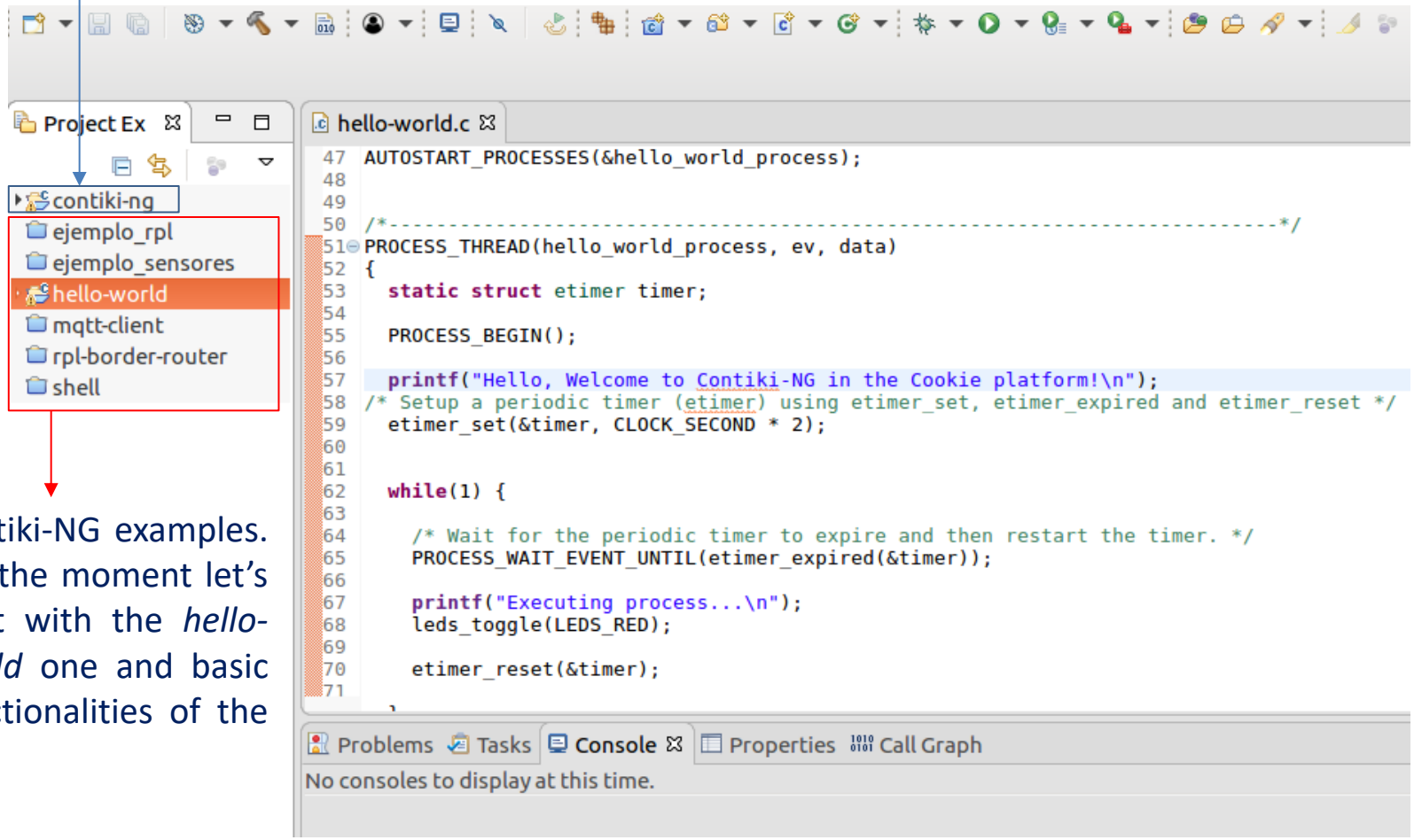
Linux and Contiki-NG Virtual Machine

- There are two main shortcuts already created in the Desktop of the Ubuntu OS:
  - *Eclipse*: Development environment that integrates the libraries and compilation capabilities to work on the Cookies.
  - *CoolTerm*: a Serial Terminal to interact with the Cookie nodes by using the UART-to-USB converter integrated in the platform.



Contiki-NG

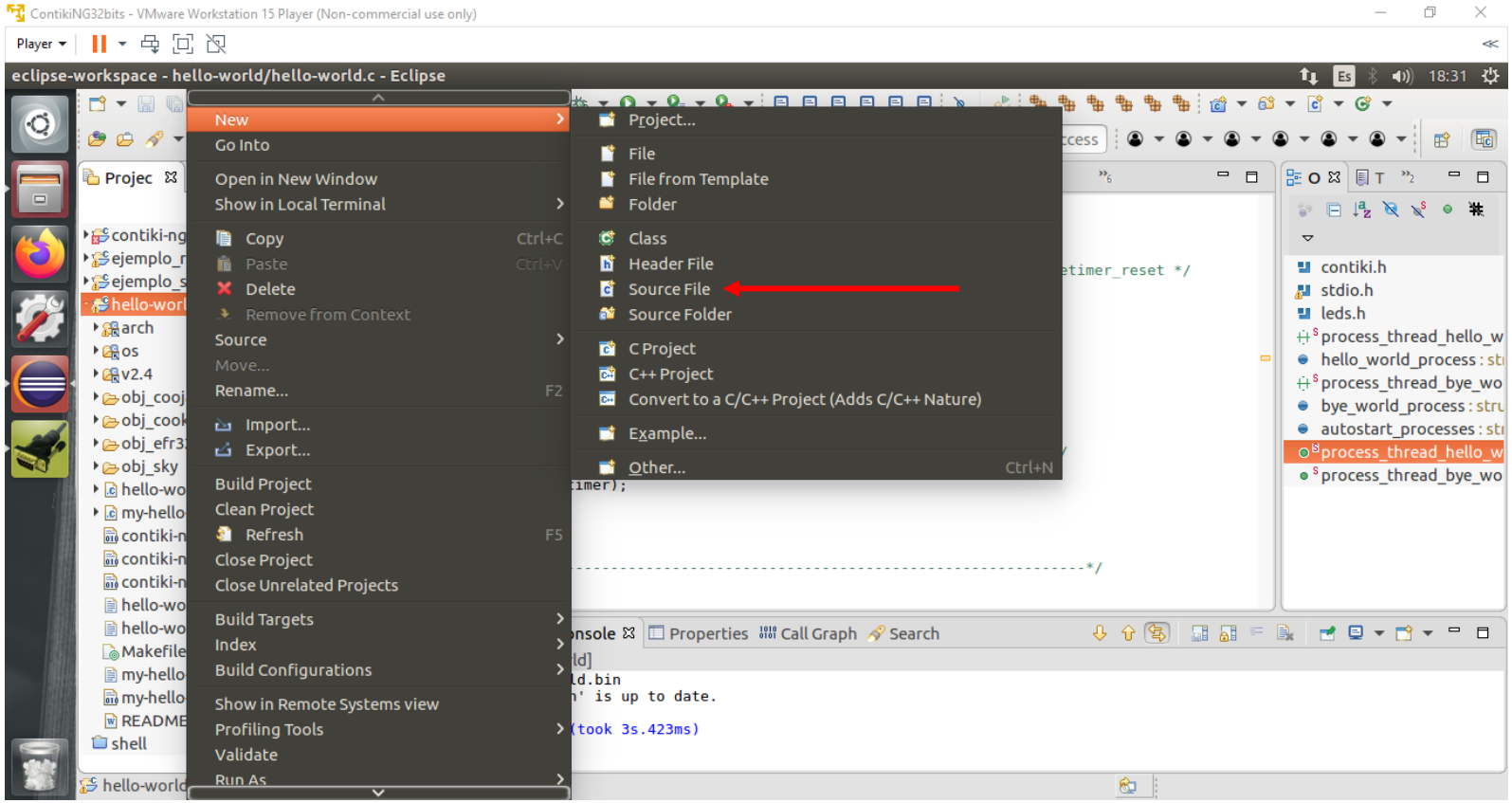
Contiki-NG Tree folder: It has to be always in the Eclipse workspace (Do not edit it).



Contiki-NG examples. For the moment let's start with the *hello-world* one and basic functionalities of the OS.

Contiki-NG

- First example: Create a new source file in the **Project "hello-world"**: Right click on the hello-world project folder > new > Source file.
- Name the new file "my-hello-world.c".



Contiki-NG

- Programming with processes

```
#include "contiki.h" /* Main include file for OS-specific modules. */
#include <stdio.h> /* For printf. */

PROCESS(test_proc, "Test process");
AUTOSTART_PROCESSES(&test_proc);
```

Variable that identifies the created process

Verbose name of the process

Auto-start after OS booting

```
PROCESS_THREAD(test_proc, ev, data)
{
  PROCESS_BEGIN();

  printf("Hello, world!\n");

  PROCESS_END();
}
```

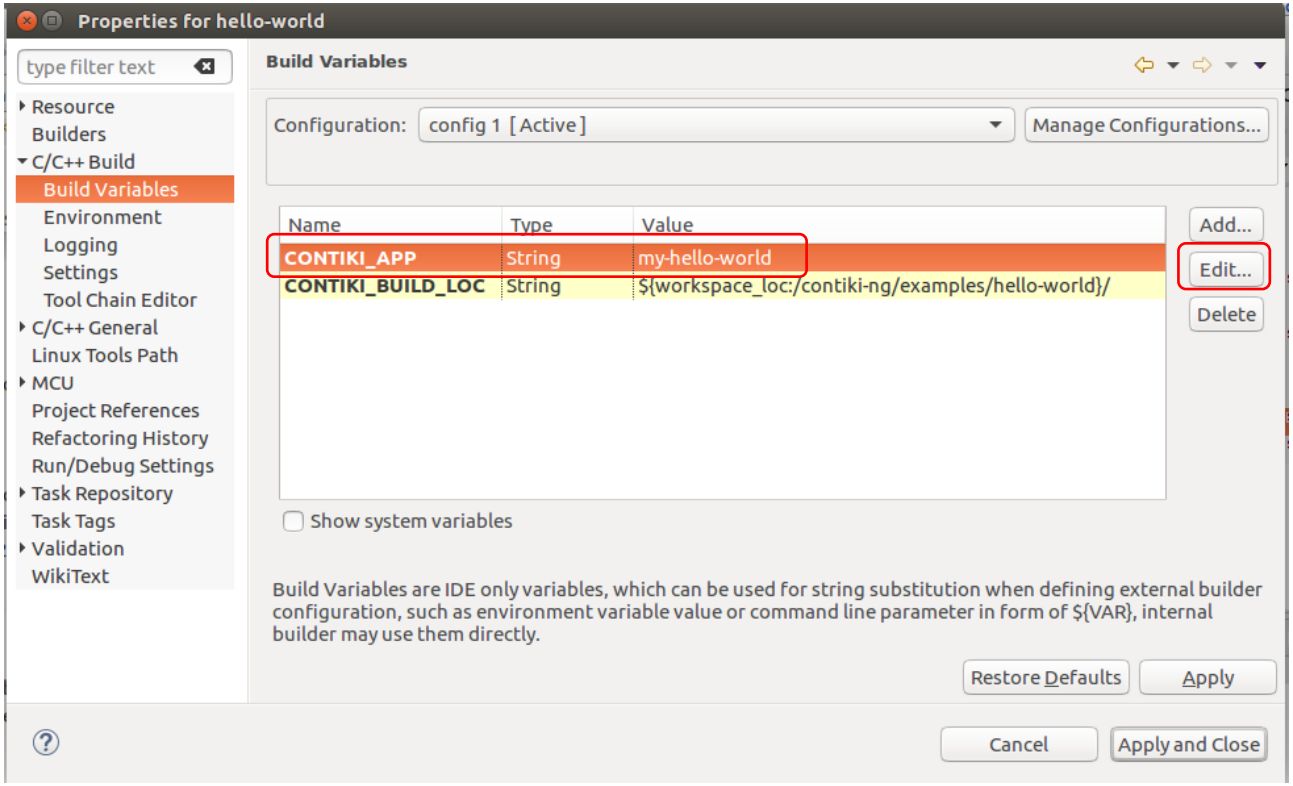
Data associated to the event

Event identifier

Process execution

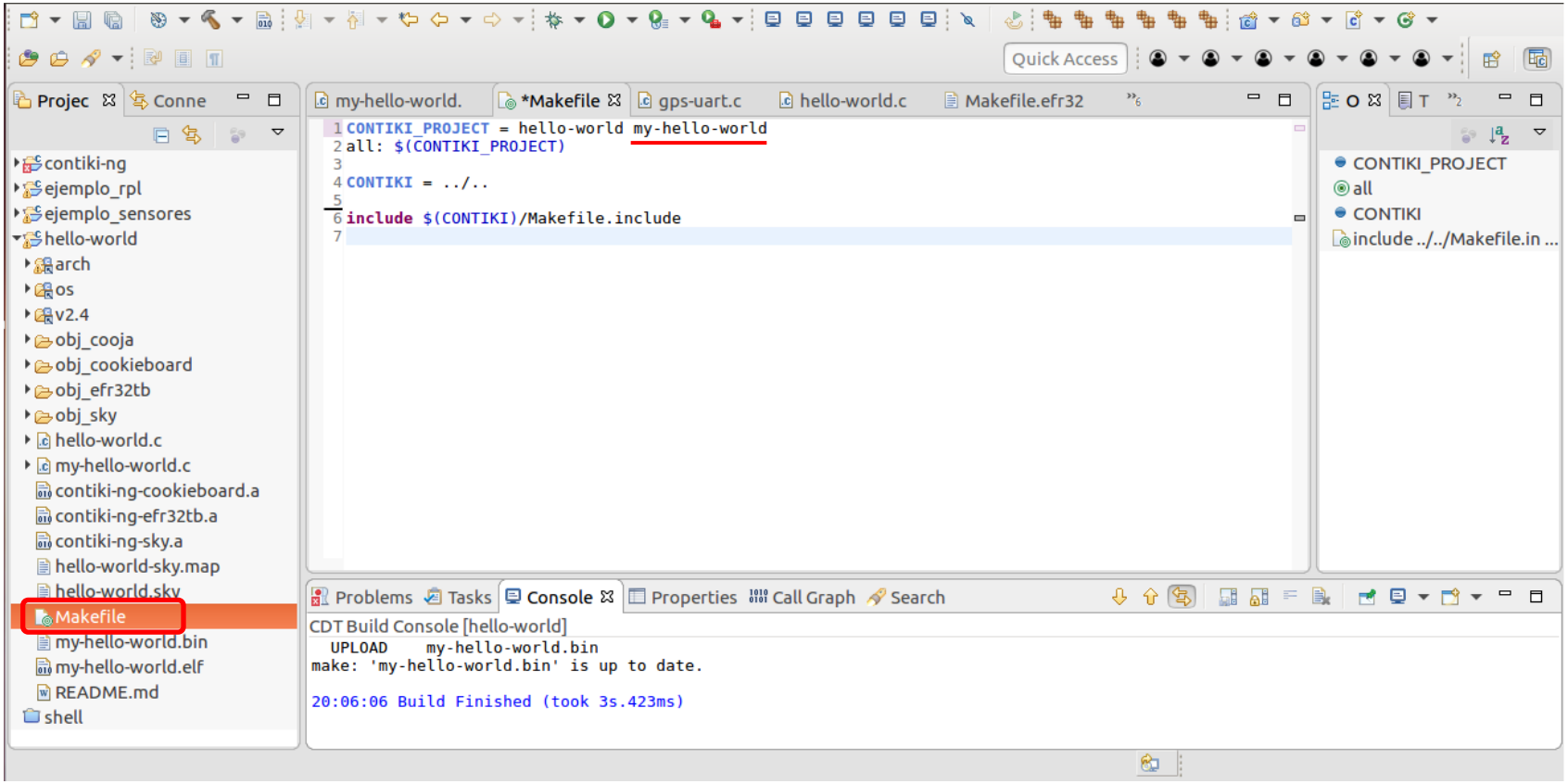
Contiki-NG

- Right click on the “hello-world” project folder > Properties
- Modify the CONTIKI_APP Variable to tell Contiki which is the target C code to be compiled and uploaded to the Cookie



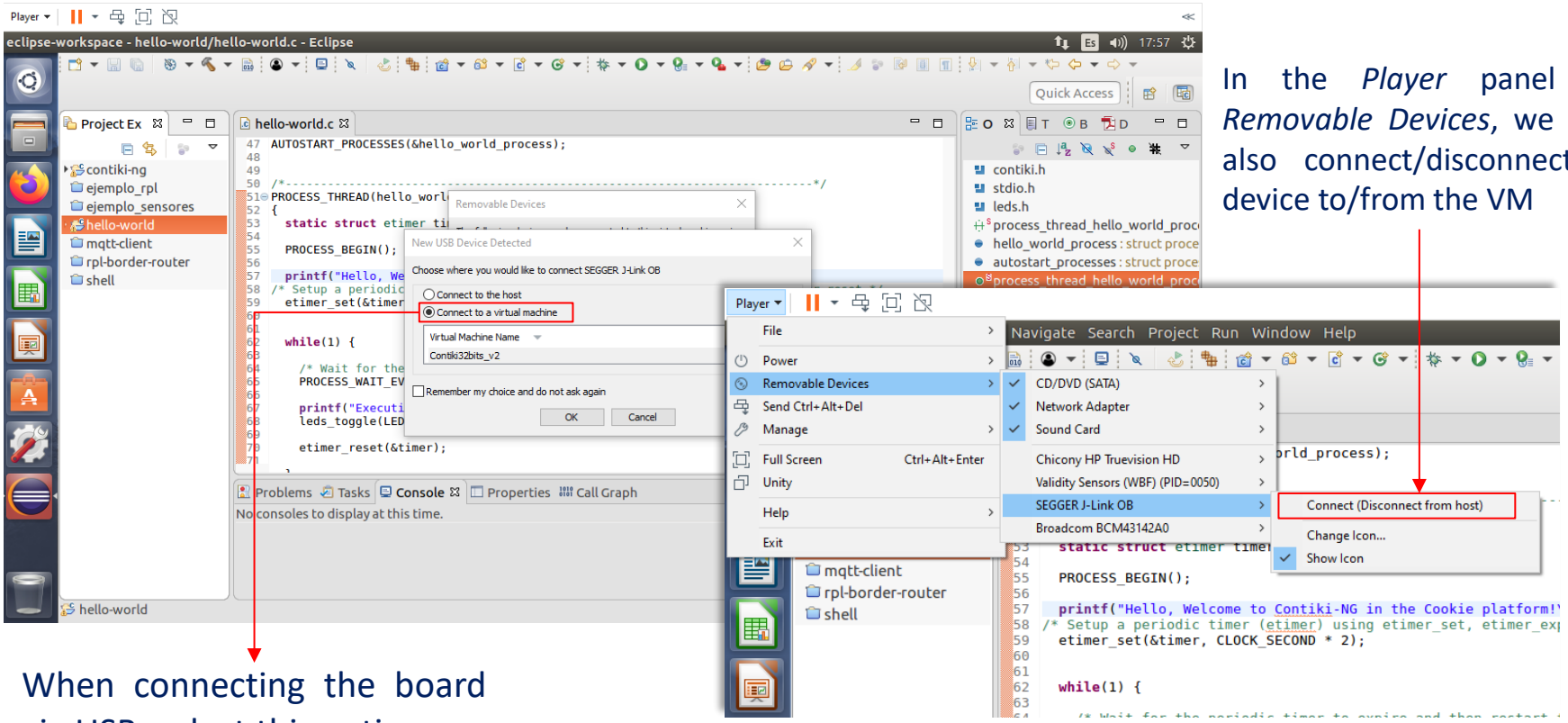
Contiki-NG

- In case user libraries are created within the project, they have to be specified into the *Makefile* for compilation (name of the C source file, without ".c"):



Contiki-NG

- To compile and upload the binary file into the Cookie, click on the **Build** button (**hammer icon**), or right click on the project folder (hello-world) > **Build Project**.
- When using the Build option, the environment is prepared to **compile** and **upload** the binary file into the Cookie. Therefore, the Debugger has to be connected and linked to the Virtual Machine:

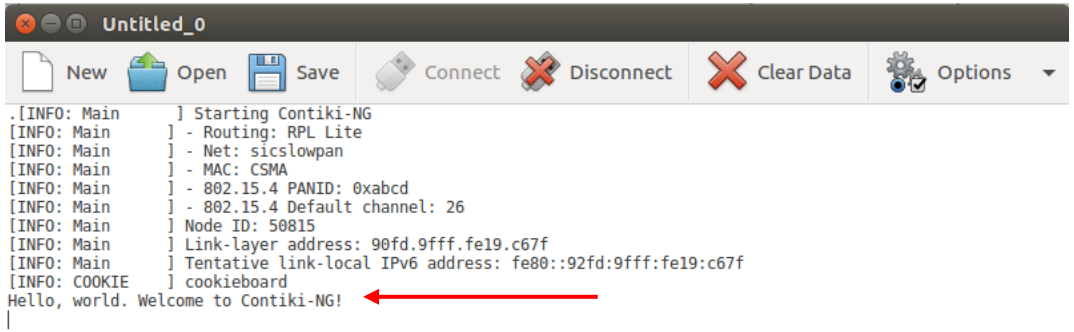


In the *Player* panel -> *Removable Devices*, we can also connect/disconnect a device to/from the VM

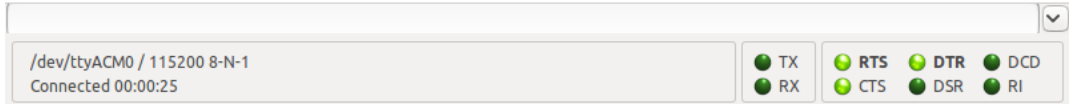
When connecting the board via USB, select this option

Contiki-NG

- Once the Cookie is programmed, let's verify the behavior of the node by using a **Serial Terminal**. Open the *CoolTerm* application located on the Desktop.
- Click on Connect (the serial port to be used will be automatically detected. If not, it can be configured in the ***Options*** panel. The port is **/dev/ttyACMx** in case of the debugger, or **/dev/ttyUSBx** in case of the USB-C; and the *baudrate* is **115200**).
- Reset the Cookie (hardware rest button of the node). The process message shall appear in the terminal:



Now, let's **turn on a LED** in the Cookie with Contiki



Contiki-NG

- Programming with processes and events.

```
PROCESS_THREAD(test, ev, data)
{
  /* An event-timer variable. Note that this variable must be static
     in order to preserve the value across yielding. */
  static struct etimer et;

  PROCESS_BEGIN();

  etimer_set(&et, CLOCK_SECOND); /* Trigger a timer after 1 second. */
  while(1) {
    PROCESS_WAIT_EVENT_UNTIL(etimer_expired(&et));
    etimer_reset(&et);
  }

  PROCESS_END();
}
```

If the condition is not met, the process yields control to the scheduler

timer: a simple timer, without built-in notification
stimer: same as timer, but in seconds
etimer: schedules events to Contiki-NG processes
ctimer: schedules calls to a callback function.
rtimer: real-time task scheduling, with execution from ISR

Let's Toggle a LED periodically:
leds_toggle(LEDS_RED); *or leds_on, leds_off functions*

Contiki-NG

- Examples to be carried out:
  - Definition of another process. 2 processes with two different behaviors.
  - Serial line events. Input characters from the Serial Terminal as events in Contiki-NG.
  - Generation of user events between processes. Broadcast events and single recipient events.
  - Use of the sensors included in the Cookie nodes.

- Exercises to be carried out (see Lab 2 homework document for details)